

LAPORAN TUGAS SISTEM OPERASI



**Dosen Pengampu Matakuliah :
Triawan Adi Cahyanto, M.K**

**Disusun Oleh :
Muhammad Gufron (2410651053)**

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
TAHUN AKADEMIK 2024/2025**

DAFTAR ISI

Isi

BAB I	4
TUJUAN PRAKITKUM	4
BAB II	5
PEMBAHASAN	5
BAB III	15
KESIMPULAN	15

KATA PENGANTAR

Puji syukur kehadiran Allah SWT yang telah memberikan rahmat dan hidayah-Nya sehingga kami dapat menyelesaikan tugas makalah yang berjudul “Memori Dalam Komputer” ini tepat waktu.

Adapun tujuan dari penulisan makalah ini adalah untuk memenuhi tugas dosen pada mata kuliah Dasar-Dasar Sistem. Selain itu, makalah ini juga bertujuan untuk menambah wawasan tentang Kerja Sama Kelompok bagi para pembaca dan juga bagi penulis.

Kami mengucapkan terima kasih kepada Triawan Adi Cahyanto, M.K

selaku dosen mata kuliah Pengembangan Diri yang telah memberikan tugas ini sehingga dapat menambah pengetahuan dan wawasan sesuai dengan bidang studi yang kami tekuni.

Kami juga mengucapkan terima kasih kepada semua pihak yang telah membagi sebagian pengetahuannya sehingga kami dapat menyelesaikan makalah ini. Kami menyadari, makalah yang kami tulis ini masih jauh dari kata sempurna. Oleh karena itu, kritik dan saran yang membangun akan kami nantikan demi kesempurnaan makalah ini.

BAB I

TUJUAN PRAKTIKUM

Tujuan dari praktikum ini adalah memahami konsep deadlock, penyebab terjadinya deadlock, serta cara pencegahannya melalui implementasi langsung di program C. Mahasiswa diharapkan mampu mengidentifikasi kondisi deadlock pada skenario multithreading, menerapkan teknik pencegahan seperti lock ordering dan try-lock, serta menguji keamanan sistem menggunakan Banker's Algorithm. Selain itu, praktikum ini bertujuan memberikan pengalaman nyata dalam menganalisis masalah sinkronisasi dan penerapan solusi yang tepat pada kasus-kasus dunia nyata, seperti proses transfer antar rekening yang dapat menyebabkan deadlock.

BAB II

PEMBAHASAN

Tugas 1: Analisis Deadlock

Jalankan program `deadlock_simple.c` dan jelaskan:

1. Mengapa program mengalami deadlock?

Program mengalami deadlock karena dua thread saling menunggu resource yang sedang dipegang oleh thread lain.

Thread 1 mengunci lock1 dulu dan kemudian berusaha mengambil lock2.
Sementara itu, Thread 2 mengunci lock2 dulu, lalu berusaha mengambil lock1.

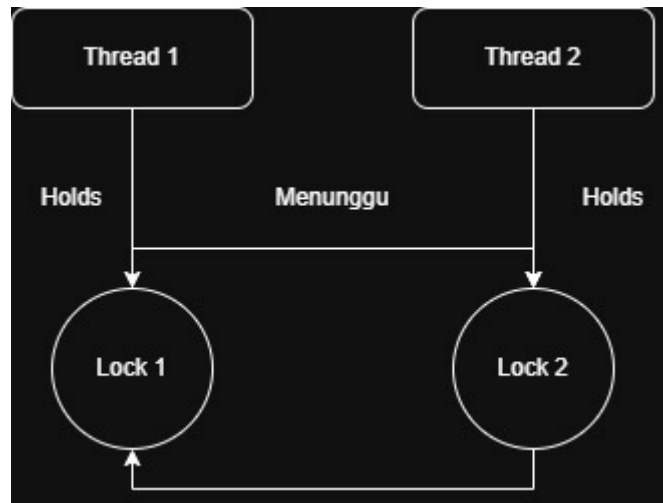
Keduanya membutuhkan lock yang sedang dipegang pihak lain, sehingga tidak ada yang bisa melanjutkan atau melepaskan lock. Akibatnya, kedua thread berhenti dan program macet.

2. Kondisi deadlock apa saja yang terpenuhi?

Deadlock terjadi apabila empat kondisi Coffman terpenuhi secara bersamaan:

- **Mutual Exclusion**
Lock1 dan lock2 hanya bisa dipegang oleh satu thread pada satu waktu.
- **Hold and Wait**
Thread 1 memegang lock1 sambil menunggu lock2.
Thread 2 memegang lock2 sambil menunggu lock1.
- **No Preemption**
Mutex tidak dapat direbut paksa oleh thread lain.
Lock hanya dilepas secara sukarela.
- **Circular Wait**
Thread 1 → menunggu lock2 → dipegang Thread 2
Thread 2 → menunggu lock1 → dipegang Thread 1

3. Gambar diagram resource allocation graph-nya



Tugas 2: Implementasi Pencegahan

Modifikasi program `deadlock_simple.c` menggunakan teknik:

1. Lock ordering

```

gufro@MSI:~/Praktikum6$ nano prevention_ordering_modifikasi.c
gufro@MSI:~/Praktikum6$ gcc prevention_ordering_modifikasi.c -o prevention_ordering_modifikasi
gufro@MSI:~/Praktikum6$ ./prevention_ordering_modifikasi
=== LOCK ORDERING (PENCEGAHAN DEADLOCK) ===

Thread 1: Mencoba mengambil Lock 1...
Thread 1: Berhasil dapat Lock 1!
Thread 2: Mencoba mengambil Lock 1...
Thread 1: Mencoba mengambil Lock 2...
Thread 1: Berhasil dapat Lock 2!
Thread 1: Selesai bekerja
Thread 2: Berhasil dapat Lock 1!
Thread 2: Mencoba mengambil Lock 2...
Thread 2: Berhasil dapat Lock 2!
Thread 2: Selesai bekerja

=== PROGRAM SELESAI ===
  
```

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
void acquire_locks_in_order() {
    printf("→ Mengambil Lock 1\n");
    pthread_mutex_lock(&lock1);

    printf("→ Mengambil Lock 2\n");
    pthread_mutex_lock(&lock2);
}
  
```

```

void release_locks_in_order() {
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
    printf("→ Lock dilepas\n");
}

void* thread1_func(void* arg) {
    printf("Thread 1: Mulai\n");
    acquire_locks_in_order();
    sleep(1);
    release_locks_in_order();
    printf("Thread 1: Selesai\n");
    return NULL;
}

void* thread2_func(void* arg) {
    printf("Thread 2: Mulai\n");
    acquire_locks_in_order();
    sleep(1);
    release_locks_in_order();
    printf("Thread 2: Selesai\n");
    return NULL;
}

int main() {
    pthread_t t1, t2;

    printf("=== PENCEGAHAN DEADLOCK: LOCK ORDERING ===\n");

    pthread_create(&t1, NULL, thread1_func, NULL);
    pthread_create(&t2, NULL, thread2_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("=== SELESAI TANPA DEADLOCK ===\n");
    return 0;
}

```

Lock Ordering adalah teknik pencegahan deadlock dengan memastikan semua thread mengambil lock dalam urutan yang sama.

Jika semua thread selalu mengambil lock1 → lock2, maka kondisi Circular Wait tidak mungkin terjadi.

2. Try-lock dengan retry

Metode ini menggunakan `pthread_mutex_trylock()` sehingga thread tidak menunggu jika lock sedang digunakan. Bila gagal, thread melepaskan lock pertama, menunggu sebentar, lalu mencoba ulang. Teknik ini menghindari deadlock karena tidak ada thread yang menunggu tanpa batas, namun dapat menyebabkan banyak percobaan ulang.

3. Bandingkan kedua metode tersebut

Lock ordering lebih sederhana, cepat, dan efisien karena tidak membutuhkan retry. Metode ini sangat efektif jika urutan lock bisa ditentukan. Sebaliknya, try-lock lebih fleksibel untuk situasi dinamis, tetapi dapat menimbulkan overhead karena thread harus mencoba berkali-kali. Keduanya mencegah deadlock, namun lock ordering lebih stabil.

Tugas 3: Banker's Algorithm

Buat program baru dengan data:

- 4 proses
- 3 jenis resource
- Available: [5, 4, 3]
- Implementasi request resource dan cek keamanan sistem


```

gufro@MSI:~/Praktikum6$ nano banker.c
gufro@MSI:~/Praktikum6$ gcc banker.c -o banker
gufro@MSI:~/Praktikum6$ ./banker
=== CEK AWAL SISTEM ===

Safe Sequence: P0 P1 P2 P3

Proses P1 meminta: [1 1 0]

Safe Sequence: P0 P1 P2 P3
Request DISETUJUI: Sistem tetap aman.

Proses P3 meminta: [2 1 1]

Safe Sequence: P1 P2 P3 P0
Request DISETUJUI: Sistem tetap aman.
gufro@MSI:~/Praktikum6$ |

```

```

#include <stdio.h>

#include <stdbool.h>

#define P 4 // jumlah proses
#define R 3 // jumlah resource

int available[R] = {5, 4, 3};

int maximum[P][R] = {
    {4, 3, 2},
    {2, 2, 2},
    {3, 2, 2},
    {5, 3, 3}
};

int allocation[P][R] = {
    {1, 0, 1},

```

```

    {1, 1, 0},

    {1, 1, 1},

    {0, 1, 1}
};

int need[P][R];

void hitung_need() {
    for (int i = 0; i < P; i++)
        for (int j = 0; j < R; j++)
            need[i][j] = maximum[i][j] - allocation[i][j];
}

bool aman() {
    int work[R];

    bool finish[P] = {false};

    int safe_seq[P], count = 0;

    for (int i = 0; i < R; i++)
        work[i] = available[i];

    while (count < P) {
        bool found = false;

        for (int i = 0; i < P; i++) {
            if (!finish[i]) {
                bool ok = true;

```

```
        for (int j = 0; j < R; j++)

            if (need[i][j] > work[j]) {

                ok = false;

                break;

            }

        if (ok) {

            for (int j = 0; j < R; j++)

                work[j] += allocation[i][j];

            safe_seq[count++] = i;

            finish[i] = true;

            found = true;

        }

    }

}

if (!found) return false;

}

printf("\nSafe Sequence: ");

for (int i = 0; i < P; i++)

    printf("P%d ", safe_seq[i]);

printf("\n");

return true;

}
```

```
bool request_resource(int p, int req[]) {

    printf("\nProses P%d meminta: [%d %d %d]\n", p, req[0], req[1], req[2]);

    for (int i = 0; i < R; i++)

        if (req[i] > need[p][i]) {

            printf("ERROR: Request melebihi NEED!\n");

            return false;

        }

    for (int i = 0; i < R; i++)

        if (req[i] > available[i]) {

            printf("Request tidak dapat dipenuhi (resource kurang)\n");

            return false;

        }

    for (int i = 0; i < R; i++) {

        available[i] -= req[i];

        allocation[p][i] += req[i];

        need[p][i] -= req[i];

    }

    if (aman()) {

        printf("Request DISETUJUI: Sistem tetap aman.\n");

        return true;

    }

    printf("Request DITOLAK: Sistem menjadi tidak aman.\n");
```

```

    for (int i = 0; i < R; i++) {

        available[i] += req[i];

        allocation[p][i] -= req[i];

        need[p][i] += req[i];

    }

    return false;
}

int main() {

    hitung_need();

    printf("==== CEK AWAL SISTEM ====\n");

    aman();

    int req1[R] = {1, 1, 0};

    request_resource(1, req1);

    int req2[R] = {2, 1, 1};

    request_resource(3, req2);

    return 0;

}

```

Program di atas mengimplementasikan Banker's Algorithm untuk mencegah deadlock pada sistem dengan empat proses dan tiga jenis resource. Program menghitung matriks need berdasarkan selisih antara maksimum dan alokasi awal. Fungsi aman() memeriksa apakah sistem berada dalam keadaan aman dengan mencari urutan eksekusi yang memungkinkan semua proses menyelesaikan tugasnya. Fungsi request_resource() digunakan untuk memproses permintaan resource dari suatu proses, dengan mengecek

apakah permintaan tidak melebihi need dan available. Jika setelah pemberian sementara sistem tetap aman, maka permintaan disetujui; jika tidak, alokasi dibatalkan.

Tugas 4: Kasus Nyata (Real Case)

Buat simulasi deadlock untuk kasus: "Dua orang ingin transfer uang antar rekening secara bersamaan

- Orang A: transfer dari rekening 1 ke rekening 2
- Oarng B: transfer dari rekening 2 ke rekening 1
- Implementasikan deadlock prevention

Deadlock pada kasus dua orang yang saling mentransfer uang terjadi karena kedua proses mengunci rekening secara terbalik (A mengunci Rekening 1 dulu, B mengunci Rekening 2 dulu). Keduanya kemudian saling menunggu sehingga sistem berhenti. Cara menyelesaikannya adalah menggunakan deadlock prevention dengan teknik lock ordering, yaitu semua proses harus mengunci rekening dengan urutan yang sama. Dalam simulasi, setiap thread selalu mengunci Rekening 1 terlebih dahulu, kemudian Rekening 2, meskipun arah transfERNYA berbeda. Dengan aturan ini, kondisi circular wait dihilangkan sehingga deadlock tidak mungkin terjadi dan kedua proses dapat melakukan transfer secara aman dan berurutan

BAB III

KESIMPULAN

Dari praktikum ini dapat disimpulkan bahwa deadlock terjadi ketika dua proses saling menunggu resource sehingga sistem tidak bisa berjalan. Teknik pencegahan seperti lock ordering dan try-lock terbukti dapat menghindari deadlock. Banker's Algorithm juga membantu memastikan permintaan resource tetap aman. Secara umum, pengelolaan resource yang tepat sangat penting untuk mencegah deadlock dalam sistem.